Bus Inf Syst Eng 67(2):171–190 (2025)
https://doi.org/10.1007/s12599-023-00846-3

RESEARCH PAPER

Putting the SWORD to the Test: Finding Workarounds with Process Mining

Wouter van der Waal · Inge van de Weerd · Iris Beerepoot · Xixi Lu · Teus Kappen · Saskia Haitjema · Hajo A. Reijers

Received: 21 February 2023 / Accepted: 23 October 2023 / Published online: 29 January 2024

Abstract Workarounds, or deviations from standardized operating procedures, can indicate discrepancies between theory and practice in work processes. Traditionally, observations and interviews have been used to identify workarounds, but these methods can be time-consuming and may not capture all workarounds. The paper presents the Semi-automated WORkaround Detection (SWORD) framework, which leverages event log traces to help process analysts identify workarounds. The framework is evaluated in a multiple-case study of two hospital departments. The results of the study indicate that with SWORD we were able to identify 11 unique workaround types, with limited knowledge about the actual processes. The framework thus supports the discovery of workarounds while minimizing the dependence on domain knowledge, which limits the time investment required by domain experts. The findings highlight the importance of leveraging technology to improve the detection of workarounds and to support process improvement efforts in organizations.

Accepted after 2 revisions by the editors of the special issue.

W. van der Waal (✉) · I. van de Weerd · I. Beerepoot · X. Lu · H. A. Reijers
Department of Information and Computing Sciences, Utrecht University, Princetonplein 5, 3584 CC Utrecht, The Netherlands
e-mail: w.g.vanderwaal@uu.nl

T. Kappen
DIT, University Medical Center Utrecht, Utrecht, The Netherlands

S. Haitjema
Central Diagnostic Laboratory, University Medical Center Utrecht, Utrecht University, Utrecht, The Netherlands

Keywords Workaround · Event data · Process mining · Pattern detection

1 Introduction

Organizational processes can be streamlined by formalizing them into standard operating procedures. However, there are many reasons why workers may not be able to strictly follow these: the prescribed procedure may not account for all subtleties of practice (Beerepoot et al. 2021), the system may work incorrectly (Tucker 2016), the software might not fit the procedure well (Ignatiadis and Nandhakumar 2009), or it may not be possible to follow the procedure if an earlier step was not performed correctly (Beerepoot and van de Weerd 2018). Workers often solve these problems by finding a new path to reach their goals. When people deviate from the procedure to reach the intended goal, we consider this a workaround in line with the definition by Ejnefjäll and Ågerfalk (2019).

In some situations, a deviation from the procedure can be inefficient, costly, or outright dangerous (Ignatiadis and Nandhakumar 2009; Ilie 2013). In other situations, workarounds may be a practical solution to problems in the procedure and can be used to improve it (Alter 2014; Beerepoot and van de Weerd 2018). Regardless of its effect, a workaround is an indication of a mismatch between a theoretical procedure and its practical execution. Knowledge of workarounds, thus, offers great potential for organizations to prevent problematic situations or improve procedures (Beerepoot and van de Weerd 2018). Using insights into workarounds, process owners can prevent the problem from occurring again or improve the corresponding procedure (Beerepoot and van de Weerd 2018). In addition, being able to identify workarounds structurally

and comprehensively would allow for the monitoring of their emergence, diffusion, and evolution, which might enable process analysts to detect and respond to new workarounds quickly. This motivates our focus on workaround detection in this paper.

Workarounds can be identified through qualitative methods, such as interviewing experts and observing their work (Beerepoot and van de Weerd 2018). Once identified, workarounds can be analyzed to gain insights into their mechanics and effects, as well as insights into the motivations of why people use them (Azad and King 2008; Ignatiadis and Nandhakumar 2009; Tucker 2016). These methods tend to require a lot of time from both the researcher and the domain expert. Furthermore, process participants may not disclose all behavior or behave differently during observations (Weinzierl et al. 2020). A set of techniques that is particularly helpful in providing insights into the real execution of processes is *process mining*. Similar to how process mining is used to solve otherwise time-consuming problems (van der Aalst et al. 2003), our focus is on the use of event logs and quantitative techniques.

In this paper, we describe and evaluate a more quantitative approach for workaround detection: the Semi-automated WORkaround Detection (SWORD) framework. The SWORD framework uses 22 patterns to identify potential workarounds from an event log. If the log contains traces, which are lists of the activities with corresponding timestamps related to a single process instance, the detection of workarounds is performed in a highly automated fashion. Thirteen patterns in the framework can be applied without domain knowledge. Only after the potential workarounds have been identified, a qualitative analysis is conducted in which domain experts are interviewed to confirm the actual occurrence of workarounds. This makes the detection of workarounds less labor-intensive than finding workarounds through interviews and observation, but also less labor-intensive than conformance checking projects, in which the normative process needs to be identified with experts before the event data can be analyzed (Rozinat and van der Aalst 2008). Also, this non-obtrusive approach can be expected to partly mitigate the change in behavior that people may exhibit when they are aware of being observed (Holden 2001).

We introduced the initial idea of the SWORD framework in an earlier study (van der Waal et al. 2022), where we only demonstrated the potential of the framework on toy examples, but did not evaluate it further. In this work, we extend that work in three ways: 1) We describe the patterns that can be applied without domain knowledge in detail, which can help during implementation of the framework; 2) We expand our workaround analysis from previous studies by including workarounds from the retail domain; and 3) We replace the demonstration by a completely new evaluation. This is important to ensure that information systems artifacts find their way to application. Since workarounds are especially common in healthcare (Röder et al. 2014), we focus on this domain. We evaluate the framework in a multiple-case study, in which we applied the SWORD patterns to data from two hospital departments. By discussing the findings with domain experts, we discovered 11 workarounds. Compared to interviews, this method required limited time and effort from domain experts.

In the remainder of this paper, we first clarify the term 'workaround' and how it relates to other subdomains in process mining in Sect. 2. We explain our research method in Sect. 3. In Sect. 4, we describe the SWORD framework and how to apply it. We validate the framework with a multiple-case study in Sect. 5. Finally, we will reflect on the results in Sect. 6 and conclude in Sect. 7.

2 Related Work

When a worker tries to reach a certain goal, they may be blocked from following the official procedure. If they can solve this by working around this block to reach their intended goal, this is considered a workaround (Alter 2014; Ejnefjäll and Ågerfalk 2019). That means that we do not consider mistakes, deception, or fraud to be workarounds, because mistakes are not goal-oriented, and fraud and deception have a different goal than the procedure altogether. There may, however, be many intentional reasons to deviate from the procedure, such as fixing a problem that was not accounted for (Ignatiadis and Nandhakumar 2009) or not being able to finish the task on time otherwise (Azad and King 2008). For clarity, we distinguish between workaround instances and workaround types. The former is an individual occurrence of a workaround, the latter describes a group of similar instances that share characteristics. For example, we would consider a worker saving data to register it at a later time to be a single workaround instance that would fall under a 'batching' workaround type. An overview of these terms is also shown in Table 2.

There are many studies considering workaround detection in various domains, such as hospitals (Azad and King 2008), nursing homes (Vogelsmeier et al. 2008), retail (-van de Weerd et al. 2019), and consulting agencies (Wolf and Beverungen 2019). While most approaches aiming to find workarounds are qualitative, we are aware of two quantitative approaches. Outmazgin and Soffer (2014) define six workaround patterns, four of which are used to detect workarounds in event logs. These patterns require strict definitions of what is considered a workaround, such as a full process model or exact thresholds for activity

duration. While these patterns can be used to detect instances of *known* workaround types, it is difficult to discover new types with them. Weinzierl et al. (2022) used a deep learning method to detect instances of seven workaround types. Neural networks require labeled training data, in this case, event log traces marked with 'workaround type x' or 'normative'. Since such a data set did not exist, the authors generated synthetic workarounds and added them to a real data set. With this approach, the neural network could recognize many of the synthetic workarounds. Since this method has only been tested on artificial data, it is unclear to what extent deep learning can be used to detect real workarounds. More importantly, in case that such a real dataset would be available to train a deep learning method, it would require the workaround types to be already known, which sidesteps the difficult task of evaluation within the domain as required for a real application. In addition, a high classification accuracy in neural networks does not necessarily mean high explainability (Ras et al. 2018), which is often important to apply results.

In the field of Process Mining, many conformance checking and deviation detection techniques have been proposed that make use of event log data to obtain insights into work processes. Conformance checking aims to evaluate to what extent executed processes conform to rules and regulations. As such, most of the conformance checking techniques require something to conform to, for example, a normative process model (Rozinat and van der Aalst 2008) or rules (Lazovik et al. 2004; Mahbub and Spanoudakis 2004). Workarounds, however, may occur without these. Deviation detection techniques are used to discover instances that differ from the norm and generally do not require rules and regulations. Many of the techniques only take the control-flow perspective into account (Mannhardt et al. 2015). However, some workarounds can only be found by using the time, resource, and data perspectives (Beerepoot et al. 2021). We will continue this multi-perspective approach to investigate if, in addition to recognizing workarounds, we can discover new workaround types using event logs.

3 Research Approach

Our study consists of two phases: a phase in which we define a list of workaround detection patterns and a phase in which we evaluate them. The two phases and their underlying activities are depicted in Fig. 1. In this section, we describe the approach followed in the two phases.

3.1 Phase 1: Defining the Workaround Detection Patterns

As described in the previous section, detecting workarounds has similarities with several approaches in process mining, such as conformance checking. Both focus on differences between the intended and actual process. To investigate these approaches, we carried out a literature review to provide an overview of process mining approaches that can be applied to workaround detection.

After an initial literature search, we selected five literature reviews as the starting point for our study. The reviews focused on several (sometimes overlapping) process mining topics, namely conformance checking (Dunzer et al. 2019), process variant analysis (Taymouri et al. 2021), predictive process monitoring (Di Francescomarino et al. 2018), deviance mining (Nguyen et al. 2016), and process mining in healthcare (Batista and Solanas 2018). The first four fields were selected based on their similarity to workaround mining. As such, techniques from these fields are more likely to be applicable in this field. The final review was included because of the high frequency of workarounds in that domain (Röder et al. 2014). In the next step, we selected relevant papers from these reviews. We used the following inclusion criteria: (1) the described approach focuses on the differences between process variants, from the control-flow, data, resource, or time perspective, and (2) the described approach uses event data for its analysis. In addition, we excluded supervised learning methods because they need a large number of labeled traces. It would require too much time to let an expert label all traces as a workaround or normative process. Note, also, that we keep to the main pattern in situations where slightly differing variants exist. For example, there are multiple versions of trace alignment; we do not

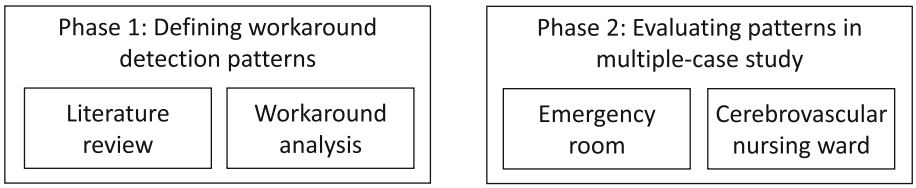

Fig. 1 The phases of our research approach

distinguish between these here. Discovered papers were also searched for new references using reverse snowballing until no new patterns came up. Overall, we analyzed 37 papers in detail and included 12 papers in our final analysis, covering 16 patterns.

Second, we carried out an analysis of 29 workaround types from retail and 81 from healthcare that we observed in earlier studies. In the retail case study, the processes and workarounds in four stores of a large Dutch supermarket chain were studied through interviews and observations (-van de Weerd et al. 2019). This resulted in 29 workarounds that were confirmed through a survey among 292 of the employees. For our analysis, we used the workaround descriptions, interview transcripts, and notes from the observations.

The healthcare workarounds were gathered from previous case studies that were carried out in five different healthcare organizations; a general hospital, two district hospitals, and two specialized care centers (Beerepoot et al. 2021). In these studies, the researchers were able to detect multiple instances of known workaround types using quantitative methods in a completely different top clinical hospital. This shows that we can expect similar behavior in other healthcare organizations. The workarounds from the healthcare studies were documented as 'workaround snapshots' and include a description of (1) the setting in which the workaround was found, (2) the workaround compared to the normative process, (3) a motivation, and (4) the expected effect of the workaround on cost, time, quality, and flexibility, and (5) transcripts of any interviews related to this workaround. To give an impression of the snapshots, we added an example snapshot in Table 1. For the transcripts, we only included a short excerpt.

Table 3 indicates which workarounds were found using literature or in either of the workaround analyses. Note that this framework is not exhaustive. As previous research is unlikely to have discovered all existing workarounds, more patterns can be added to the framework by investigating other departments, organizations, domains, or research fields. While not required, using additional domain knowledge for an initial extension of the framework may provide additional insights.

Using the comparison between the workaround and the normative process, we determined if and how each workaround could be monitored using (event log) data. Similar patterns were grouped together. This workaround analysis yielded another six patterns.

By combining the patterns from the literature study and the workaround analysis, we discovered 22 workaround

Table 1 An example of a complete workaround snapshot

Setting	The researcher is in the orthopedics and surgery ward. Several nurses are working on computers. Patients and other nurses sometimes walk by. The researcher observes the nurses and asks them what they are doing
Workaround compared to the normative process	A patient arrives in the nursing ward after undergoing an operation. A physician is responsible for registering the medication information of this patient in the hospital information system but does not always do this. Upon arrival in the nursing ward, the nurses find out that the medication information is absent or incomplete. Nurses will try to call the physicians to recover this information. Often, the nurse has to wait until the physician is available to prescribe the medication. The nurse subsequently enters the medication information ad-hoc. This enables the nurse to administer the medication
Motivation	Physicians are often busy and perform multiple operations per day. They are also not aware of how their workaround hinders the nursing practice. The nurse feels that the patient should no longer wait for the medication and decides to proceed with the workaround
Effect	*Costs: negative.* Short-term positive because the nurse carries out tasks of the (assistant) physician. Long-term negative, since the process takes longer which increases costs (see time)
	Time: negative. In the short-term, it costs the (assistant) physician less time and the nurse extra time. In the long-term, it will take more time since an ad-hoc medication registration is only for one time and has also to be verified by a physician. In addition, it takes time to call the physician
	Quality: negative. Not registering important information might lead to cases where the patient does not receive the right medication or not at the right time
	Flexibility: negative. The nurse cannot administer the medication directly but has to call the physician. The physician is disturbed by calls during moments it might not be suitable
Transcripts	Transcripts of interviews with one team leader and two nurses. Excerpt: "It takes the entire day to get it all in there before someone can get their medicine. You keep administering it ad-hoc... because ad-hoc is only for a single time. While if the physician would just verify it, it would be in there automatically and he would not need to be called all the time." (Nurse)

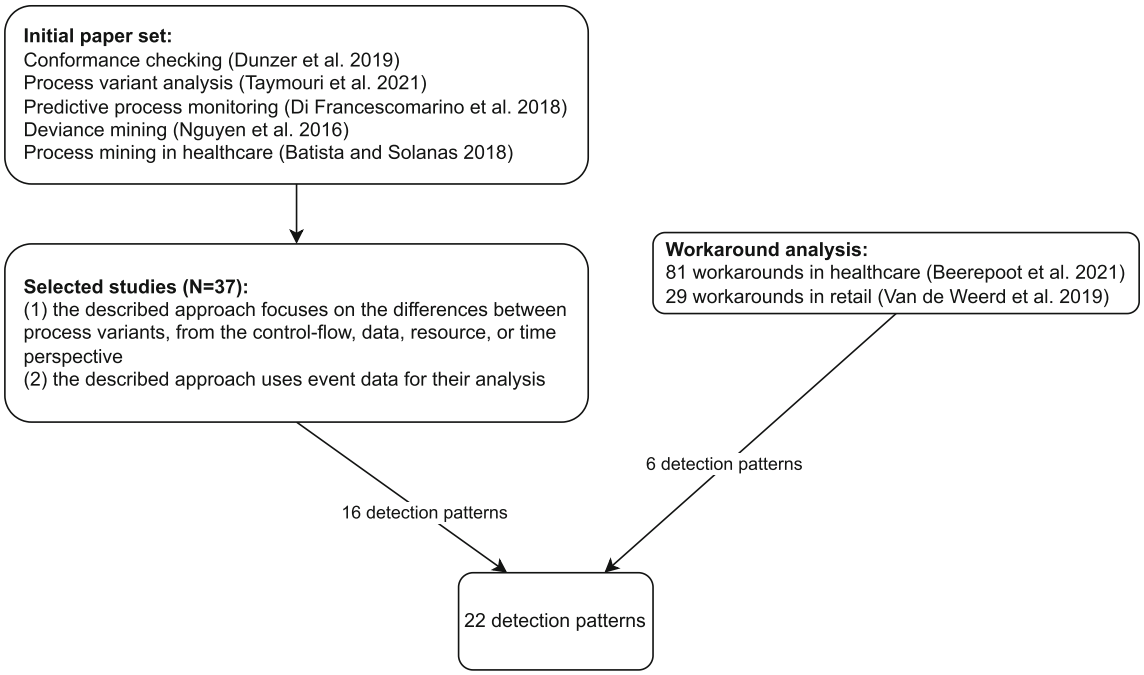

Fig. 2 Phase 1 of our study: The construction of the patterns in the SWORD framework

detection patterns. Figure 2 shows a summary of this phase.

3.2 Phase 2: Evaluating Patterns in a Multiple-Case Study

In the second phase of our study, we evaluated the list of workaround detection patterns. We followed a two-step approach for this. First, we wanted to establish whether the data necessary to detect a pattern was stored in the Hospital Information System (HIS). Therefore, we analyzed the data structure of the tables in which the event data of the relevant processes and workarounds are stored. For each pattern, we searched for a table containing the required data to identify it. At this point, we considered if we could use that data to discover new workaround types, or if the pattern relied on specific knowledge and could only be used to monitor known workarounds. This activity resulted in an overview of data requirements per pattern.

Second, we conducted a multiple-case study at two different wards of the University Medical Center Utrecht (UMCU). In this study, we focus on discovering new workaround types, so we only consider the patterns that can be used for this. Since workarounds in healthcare are common (Röder et al. 2014), a case study at this hospital is a suitable way of evaluating the framework by providing the opportunity to both find known healthcare workaround instances (Beerepoot et al. 2021) and discover new workaround types. The details of the approach for this phase can be found in Sect. 5.

4 SWORD Framework

In this section, we present the SWORD framework. Table 2 shows the key terms we use. We start by analyzing the patterns we found using the literature review and workaround analyses. Specifically, we show how these patterns are applied in existing studies. We then take a further look into what data is required to use each pattern. Finally, we provide a detailed description of the patterns that can be applied regardless of available domain knowledge and explain how to rank traces using the patterns.

4.1 Workaround Detection Patterns–Literature Review

Researchers in process mining fields other than workaround mining, such as conformance checking, deviance mining, predictive process monitoring, and process performance analysis, have developed patterns to detect differences between process variants. Since we can recognize different workarounds using varying perspectives, we have structured our review the same way. We start with the control-flow perspective and continue with the data, resource, and time perspective.

The control-flow perspective is used in most process mining fields, often by comparing traces to process models (Beerepoot et al. 2021; Borrego and Barba 2014; Rebuge and Ferreira 2012). Alternatively, some approaches analyze traces to check for activities that should never co-occur (Borrego and Barba 2014) or that should occur close to specific others (Lu et al. 2016). Some individual events

Table 2 Explanation and examples of the key terms in this paper

Key term	Explanation	Example
Workaround instance	An individual occurrence of a workaround	Nurse A prescribes medication ad-hoc instead of waiting for a physician
Workaround type	A group of similar workaround instances that share characteristics	Nurses sometimes prescribe medication instead of waiting for a physician
Workaround detection pattern	A pattern that may be used to monitor or discover workarounds	An activity is executed by an unauthorized resource

can already be interesting to monitor (Bolt et al. 2018; Bose and van der Aalst 2013; Nguyen et al. 2016). Repeated behavior can also be an indication something is going wrong. We can look at how often activities repeat (Borrego and Barba 2014; Bose and van der Aalst 2013; Nguyen et al. 2016; Swennen et al. 2016) or whether a trace contains loops (Bose and van der Aalst 2013; Lo et al. 2009; Nguyen et al. 2016; Swennen et al. 2016; Wynn et al. 2017).

The data perspective is mostly used in the conformance checking field, where the values of data objects can be simply evaluated (Beerepoot et al. 2021; Borrego and Barba 2014; Bose and van der Aalst 2013; Burattin et al. 2016; de Leoni and van der Aalst 2013; Taymouri et al. 2021). If they deviate too much, this can indicate unintended behavior in the trace. Alternatively, the exact value might not be important, but the value should not change during the trace (Borrego and Barba 2014; Burattin et al. 2016; de Leoni and van der Aalst 2013).

The resource perspective shows similar patterns. Some activities should always be executed by a specific resource or the resource needs to stay the same during the trace (Beerepoot et al. 2021; de Leoni and van der Aalst 2013). For example, certain medications should only be prescribed by a physician. While not a workaround detection pattern in itself, earlier mentioned patterns can also be used using a resource as case ID. In this way, we can investigate the behavior of resources. For example, if we do so and notice a resource is repeating the same activity, something might be going wrong (Swennen et al. 2016).

We can find deviating patterns using the time perspective in multiple process mining fields. From a conformance checking viewpoint, some activities may need to be executed at a specific time (de Leoni and van der Aalst 2013). Process performance analysis naturally takes time into account too: The time of activity since the start of the trace can predict the entire performance of the entire trace (Bolt et al. 2018). Multiple fields distinguish between process variants by looking at the time between activities (Burattin et al. 2016; de Leoni and van der Aalst 2013; Swennen et al. 2016; Wynn et al. 2017), the duration of a single activity (Swennen et al. 2016; Taymouri et al. 2021; Wynn et al. 2017), or the total time of a trace (Taymouri et al. 2021).

Each study listed in this section describes at least one aspect that is used to distinguish traces in the various process mining fields, which we translate into more general patterns. Table 3 shows an overview and description of all 22 workaround detection patterns, together with the studies they were described in and both workaround analyses we performed. As can be observed from the table, we found several workaround detection patterns in the healthcare data that have not been documented in literature. We did not find any new workaround detection patterns in the retail data.

4.2 Workaround Detection Pattern Analysis

To investigate what data is required to detect each workaround detection pattern, we have used the data structure of HiX.[1] Using this specific HIS, we explored which columns in the data we would need to use to find each pattern. In total, there were 386 tables, where we mostly used one of the 12 tables containing information concerning measurements. While we used a specific implementation of an HIS, we used the data structure as a guide to investigating what information would be required. Other information systems structure data differently, but this does not affect the data requirements for the patterns. Therefore, this analysis is relevant for any other information system, even if it is unrelated to healthcare.

To use these patterns, it should always be clear to which trace an event belongs. Depending on the available data, we can use timestamps (e.g., by grouping events that are temporally close), specific activities (e.g., a trace starts with logging in and ends with logging out), or dedicated case IDs. Table 4 shows an overview of the required data. We distinguish between four data types that may be

[1] https://www.chipsoft.com/solutions/ (Last accessed 15 November 2023)

Table 3 Workaround detection patterns

	Workaround detection pattern	Explanation	References	Healthcare Workaround	Retail Workaround
Control-flow	Occurrence of an activity	A specific activity occurs	Bolt et al. (2018); Bose and van der Aalst (2013); Nguyen et al. (2016)	X	X
	Occurrence of recurrent activity sequence	A recurrent activity sequence occurs within a trace	Bose and van der Aalst (2013); Lo et al. (2009); Nguyen et al. (2016); Swennen et al. (2016); Wynn et al. (2017)	X	X
	Activity frequency out of bounds	There is a deviation in the frequency of an activity within a trace	Borrego and Barba (2014); Bose and van der Aalst (2013); Nguyen et al. (2016); Swennen et al. (2016)	X	X
	Occurrence of activities in an order different from process model	The order of activities in a trace is other than in a predefined process model	Borrego and Barba (2014); Rebuge and Ferreira (2012)	X	X
	Occurrence of mutually exclusive activities	Specific activities occur that are mutually exclusive within a trace	Borrego and Barba (2014)		X
	Occurrence of unusual neighboring activities	An activity is directly followed by an activity other than usual	Lu et al. (2016)	X	
	Occurrence of directly repeating activity	An activity is immediately repeated within a trace		X	
	Missing occurrence of activity	A specific activity is missing in the trace		X	X
Data	Data object with value outside boundary	The value of a data object deviates from the usual values	Borrego and Barba (2014); Bose and van der Aalst (2013); Burattin et al. (2016); de Leoni and van der Aalst (2013); Taymouri et al. (2021)	X	X
	Change in value between events	Data values change unexpectedly between events	Borrego and Barba (2014); Burattin et al. (2016); de Leoni and van der Aalst (2013)	X	X
	Specific information in free-text fields	Information is logged in free-text fields instead of dedicated fields		X	X
Resource	Activity executed by unauthorized resource	An activity is executed by a resource other than those authorized	de Leoni and van der Aalst (2013)	X	
	Activities executed by multiple resources	Activities within the same trace are executed by multiple resources	de Leoni and van der Aalst (2013)	X	X
	Activities executed by a single resource	Activities within the same trace are all executed by the same resource	Swennen et al. (2016)	X	
	Activity frequency out of bounds for a resource	There is a deviation in the frequency of an activity within a trace for one resource compared to other resources		X	X
	Value frequency out of bounds for a resource	There is a deviation in the frequency of a data value within a trace for one resource compared to other resources		X	
Time	Occurrence of activity outside of time period	An activity occurs outside of the usual time period	de Leoni and van der Aalst (2013)	X	
	Delay between start of trace and activity is out of bounds	There is a deviation in the delay between the start of the trace and the time of an activity	Bolt et al. (2018)	X	
	Time between activities out of bounds	There is a deviation in the time between activities	Burattin et al. (2016); de Leoni and van der Aalst (2013); Swennen et al. (2016); Wynn et al. (2017)	X	
	Duration of activity out of bounds	There is deviation in the duration of an activity	Swennen et al. (2016); Taymouri et al. (2021); Wynn et al. (2017)	X	
	Duration of trace out of bounds	There is a deviation in the duration of a trace	Taymouri et al. (2021)	X	
	Delay between event and logging is out of bounds	There is a deviation in the delay between the time of event and time of logging		X	

required: activity, time, data, and resource. Each pattern may require a different level of quality for these types:

- Some patterns need *specific* data. This data must be known beforehand. For example, a data field may require a certain value. Because of the huge number of data fields, it is not feasible to find these values automatically. We cannot find new workaround types with these patterns, only monitor discovered ones.
- We generally require *high-quality* data. Activity names should be distinguishable from each other, timestamps need to determine when an event happened, data needs to be complete, or we need to know which resource executed an event. The exact requirements differ per process. For example, in one case, "register measurement" is sufficient, but in another, we need the measurement type.
- For the time dimension, *low-quality* data may be sufficient if high-quality is not available. In that case, timestamps only need to be precise enough to determine a correct event order.
- Some data types are *not needed* to find a pattern. For example, if we investigate if the right resource performed an activity, we do not require timestamps.

4.3 Patterns

Nine of the workaround detection patterns presented in Table 4 require specific domain knowledge concerning the workaround to use. Therefore, these patterns can only be employed to find new instances of known workaround types, but not to discover new types. These patterns can be of use when monitoring known workarounds. For example, in the Emergency Room, a patient should only be discharged after at least two pain scores have been obtained. The '*Missing occurrence of activity*' may be applied to check if the workaround type occurs in a dataset by counting if the number of times this activity occurs in a trace is at least two. However, applying this pattern without the domain knowledge results in a long list of activities that are not always executed in the same frequency, which is perfectly fine in most cases. After all, not all patients require the same care. For this paper, we are interested in discovering new workaround types, so we will further describe the 13 patterns that can be used for this. We provide an example of the patterns based on previously discovered workarounds and then further define them.

4.3.1 Pattern 1: Occurrence of Recurrent Activity Sequence

A repeating activity sequence signals that the same task is performed multiple times. While some tasks consisting of multiple activities are expected to be performed regularly, other sequences should not. For example, when a physician writes a discharge note for a patient, they first have to select if and what medication is still necessary and then print the letter. If the quantity of the medication is not correct on the printed letter, due to a system error, the physician needs to repeat the process, which results in the same activity sequence showing in the event log multiple times: $<..., open\ note, edit, print, ..., open\ note, edit, print, ...>$.

Finding non-trivial recurrent activity sequences, such as $<..., a, b, c, ..., a, b, c, ...>$ is an advanced pattern recognition problem that is beyond the scope of this paper. We refer to other studies focusing on this exact question, such as one by Bose and van der Aalst (2013).

4.3.2 Pattern 2: Activity Frequency out of Bounds

If an activity occurs either more or less often than usual, this may signal that people are working around a problem. For example, when patients have a health issue that requires a multidisciplinary approach, they should be linked to a specialist for each field. In some cases, a physician does not register their colleague as a co-practitioner but instead requests multiple peer consultations, which is usually used for incidental occurrences. A high frequency of peer consultations would make this observable in the data. Patients receive the required expertise, although not in an intended way.

We implement this pattern by counting how many times an activity occurs in each case. This results in a mean and standard deviation for every unique activity. If an activity frequency deviates strongly from the mean, either higher or lower, in a trace, it is considered more likely to contain a workaround.

4.3.3 Pattern 3: Occurrence of a Directly Repeated Activity

Activities that occur multiple times in succession indicate repeated behavior. This can indicate multiple people executing the same task or a single person repeatedly attempting it. For example, when patients get discharged from the hospital, they may get a discharge letter containing information for their follow-up. Nurses often have to wait until a physician has finished this letter. If they expect that a physician will not notify them after having written the letter, they repeatedly check the same page in the HIS, without performing any other task within the system between the checks. This final activity could be observed as directly repeating in an event log.

This pattern can be checked by simply counting if an activity occurs directly after the same one in the (time-sorted) event log. For example, in a case with

Table 4 Workaround detection patterns with the required data

	Workaround detection patterns	Activity	Time	Data	Resource
Control-flow	**Occurrence of activity*	*Specific*	–	–	–
	Occurrence of recurrent activity sequence	High quality	Low quality	–	–
	Activity frequency out of bounds	High quality	–	–	–
	**Occurrence of activities in an order different from process model*	*Specific+ Process model*	*Low quality*	–	–
	**Occurrence of mutually exclusive activities*	*Specific*	–	–	–
	**Occurrence of unusual neighboring activities*	*Specific*	*Low quality*	–	–
	Occurrence of directly repeating activity	High quality	Low quality	–	–
	**Missing occurrence of activity*	*Specific*	–	–	–
Data	**Data object with value outside boundary*	–	–	*Specific*	–
	Change in value between events	–	Low quality	High quality	–
	**Specific information in free-text fields*	–	–	*Specific*	–
Resource	Activity executed by unauthorized resource	–	–	–	High quality
	Activities executed by multiple resources	High quality	Low quality	–	High quality
	Activities executed by a single resource	High quality	Low quality	–	High quality
	**Activity frequency out of bounds for a resource*	*High quality*	–	–	*Specific*
	**Value frequency out of bounds for a resource*	-	–	*High quality*	*Specific*
Time	Occurrence of activity outside of time period	High quality	High quality	–	–
	Delay between start of trace and activity is out of bounds	High quality	High quality	–	–
	Time between activities out of bounds	High quality	High quality	–	–
	Duration of activity out of bounds	High quality	High quality	–	–
	Duration of trace out of bounds	–	High quality	–	–
	Delay between event and logging out of bounds	–	High quality	High quality	–

Those marked with * can only be used to monitor known workaround types.

$< \ldots, x, x, \ldots, x, x, x, \ldots >$, we count three repetitions of x, i.e., an x directly succeeded by another x without any other event in between.

4.3.4 Pattern 4: Change in Value Between Events

Aside from the well-known activities, timestamps, and resources, there are often many data fields linked to a process. Many values here are expected to change over the course of a trace. For example, heart rate measurements can change every measurement. Other data tends to be more static; for example, a patient's height stays constant during hospital stays.

One example we have seen where data fields changed where they were expected to stay constant, concerned appointment times that were planned in advance. In some cases, due to either user or system errors, multiple patients were scheduled to visit the same physician at the same time. These could be solved by updating the times for the patients. If we monitor this data over time, we would see that the scheduled appointment times change during the workaround but would stay static otherwise.

We can use this pattern to identify workarounds for each data field by counting the number of times it changes within a single trace. Highly dynamic data fields would have a high mean on this count, while the mean of the less dynamic data fields would be close to zero. We can then evaluate a trace by comparing the number of times each data field changed to the average of that particular field.

4.3.5 *Pattern 5: Activity Executed by Unauthorized Resource*

Some activities should be executed by specific resources or resources with specific rights. For example, medication for a patient should generally be prescribed by a physician. In the ER, some nurses are also allowed to prescribe medication for emergency patients when there is no time to consult a physician. This authority to prescribe medication can also be used in non-urgent cases to save time. After all, if nurses do it themselves, they do not have to wait.

To investigate this pattern, we can use resource roles (such as nurse, physician, or administrator) and check which roles usually perform each activity. This results in a distribution of the roles for each unique activity. For a single trace, we can use this pattern by checking the probability that the executing resource for an activity is normal by comparing it to this distribution. If an activity is usually executed by a different resource type, we would further investigate the reason for this.

4.3.6 *Pattern 6: Activities Executed by Multiple Resources & Pattern 7: Activities Executed by a Single Resource*

When activities are performed by more resources than usual, it may be a sign that people are doing work for others. In Fig. 3, for example, nurses on a day shift are sometimes too busy to register the vital signs they measure. The night shift can support them by registering the measurements later, using the paper notes of the day shift. This results in two resources being involved in the trace, instead of one.

Conversely in Fig. 4, the involvement of fewer resources can also indicate workarounds. Before administering certain medications, two nurses should check the correct dosage. Since the check is done outside the system, they should register it manually. In some cases, one nurse registers both checks. While there are two resources involved, only one would be visible in the data.

We check this pattern by counting the number of resources involved in a trace. This results in a mean and standard deviation of the number of resources involved in a trace. The higher the deviation from the mean for the number of resources in a trace, the more likely a workaround may be. Trace length differs strongly between patients. Some visit the hospital for a few hours, while others are there for weeks.

We observed an issue with these two patterns: if there are more events in a trace, more resources are generally involved. To account for this, we have a second instance for this pattern. We divide the number of resources by the number of events in the trace. In this way, we see how many resources were involved per activity. We are still able to determine a mean and standard deviation for each trace.

4.3.7 *Pattern 8: Occurrence of Activity Outside of Time Period*

Some activities are more common during certain times. If resources logged measurements directly after taking them, we would expect to see a relatively constant occurrence of the event over time. It would be higher during the day shift compared to the night shift, but within shifts, the changes in frequency would be limited. Similar to Fig. 3, a commonly used workaround is that while the measurements are taken on time, the logging is done in batches at a later time, especially during the end of a shift. If this happens frequently, we can see that logging an event occurs more often during those times.

Since this pattern is strongly based on specific workarounds occurring at the same time, we do not rank individual traces. Instead, we create a graph showing when specific activities occur, such as Fig. 5. A taller-than-expected bar in the graph shows events occurring at a non-standard time.

4.3.8 *Pattern 9: Delay Between Start of Trace and Activity is Out of Bounds*

Some activities tend to occur at the same time since the start of the trace. For example, in Fig. 6, a patient in the emergency room should be seen by a nurse near the

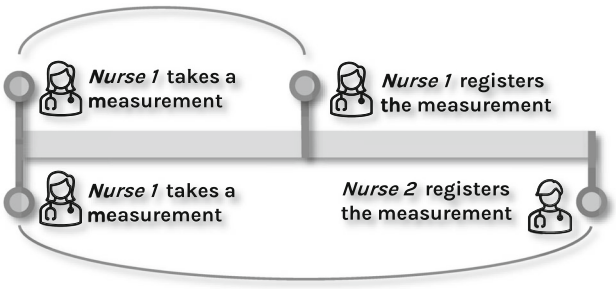

Fig. 3 In the normative process (top), nurse 1 takes a measurement and registers it directly afterward. In the workaround (bottom) nurse 2 registers the measurement at a later time

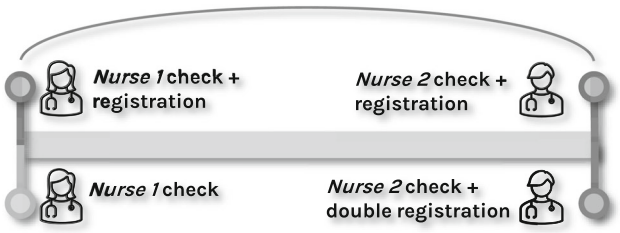

Fig. 4 In the normative process (top), nurses 1 and 2 check the medication and register it themselves. In the workaround (bottom), nurse 1 checks the medication, but does not register it. then nurse 2 checks it and registers it for both nurses

beginning of the visit. If this happens near the end of the trace, either the patient waited for a long time, or the registration was delayed, which could indicate a workaround.

We can detect this pattern by calculating the time since the first event in the trace for every unique activity. Based on these values we calculate the mean and standard deviation per activity. If the time since the start of a trace differs strongly for an event compared to the mean, we can suspect a workaround.

4.3.9 Pattern 10: Time Between Activities Out of Bounds

In many processes, the time between different events is comparable. In Fig. 4, for example, if multiple nurses check the medication, this takes a certain amount of time. In the workaround process, it is performed by a single resource, resulting in less time between the checks. With Pattern 6 we check the number of resources to detect the workaround, but with this pattern, we focus on the time between the medication checks.

We can detect this by calculating the time between every subsequent event in a trace. This results in a mean and standard deviation of the time between events for a process. If there is a long time between two events compared to the mean, this suggests a workaround.

4.3.10 Pattern 11: Duration of Activity Out of Bounds

To prevent data leaks, many organizations apply the Clear-Screen policy; if you are not at your desk, you should lock your screen or log out, so no one will be able to access your system. Logging in after returning is often seen as a waste of time, so people do not always log out when leaving their computers. When investigating the usage time of the system, this would show as a continuous multi-hour activity, as opposed to multiple shorter instances, separated by breaks.

If the data shows the duration of each activity, we can simply calculate the average and standard deviation of it for each unique event. If a trace then shows an exceptionally long or short duration for an activity, we are interested in investigating that trace further.

4.3.11 Pattern 12: Duration of Trace Out of Bounds

Many traces take roughly the same amount of time. If the duration of a full trace is significantly longer or shorter, this can indicate a workaround. We can see an example in Fig. 3: one nurse takes a measurement and another nurse registers it later. The normal process finishes shortly after the measurement, while the workaround does not finish until a later shift.

If there are only timestamps available for the start of events, we can detect this pattern by calculating the duration between the first and the last event for a trace. If ending timestamps are available too, the results will be more precise. In either case, this results in a mean and

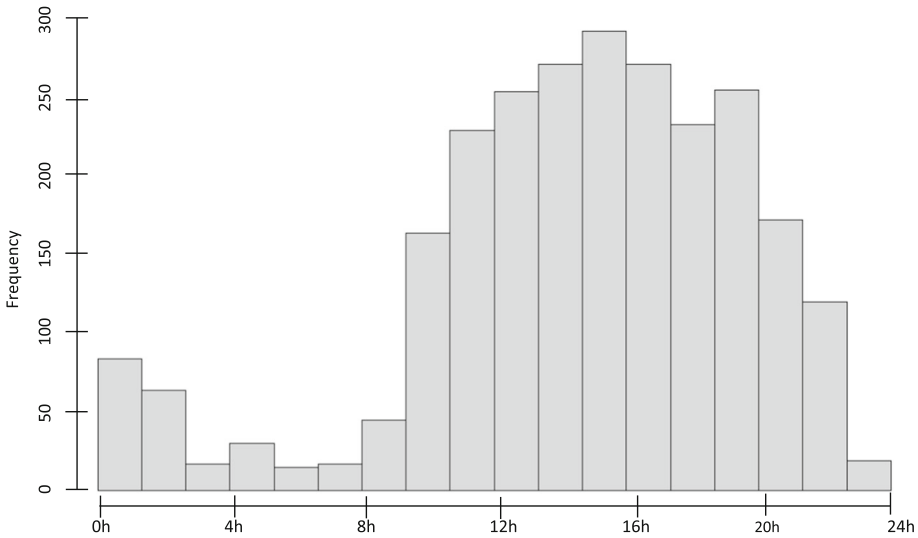

Fig. 5 Timestamps of all orders in the ER. The distribution follows the usual patient load, except for the taller bar before 20 h. This coincides with a shift change. In addition, there is a large difference between the final bar before 24 h and the first bar before 0 h, where 0 h is a common time to round to if the exact time of measurement cannot be remembered

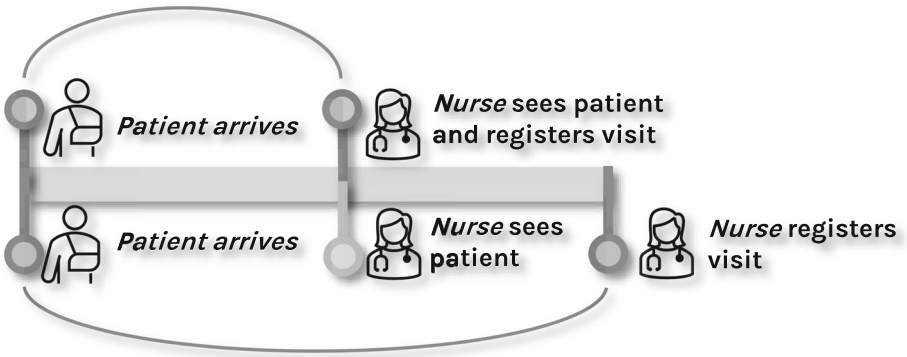

Fig. 6 In the normative process (top), the nurse registers her visit to the patient immediately. In the workaround (bottom), she visits the patient at the same time but registers it later

standard deviation of the duration of all traces. A trace that takes a lot more time compared to the mean points to a potential workaround.

4.3.12 *Pattern 13: Delay Between Event and Logging is Out of Bounds*

In most cases, an event should be logged shortly after it takes place according to official procedures. Otherwise, details about the event may be lost or other healthcare providers are not able to respond timely. Depending on the event, different time delays may be acceptable. For example, it is acceptable to delay a routine length measurement for a patient, because it does not serve as a warning score. A temperature measurement is more important to log quickly since a fever can indicate acute health issues.

If there is a dedicated timestamp available for both the measurement and the logging, we can determine the mean and standard deviation of the logging delay per activity. If an event has a logging delay that differs strongly from the mean, this is an indication of a workaround.

4.4 Ranking

In the final step of the application of the SWORD framework, we determine if there is a potential workaround in a trace using the patterns. For each pattern that can be applied to the event log in line with Table 4, we can create a ranking that determines how strongly each trace deviates from the norm. Table 5 shows an example of such a ranking. Only patterns 1, 3, and 8 are exceptions to this approach.

Patterns 2, 5, 9, 11, and 13 result in a value for each event in a trace. We determine the mean for each unique activity and compare the value for each event in a trace to it using a Z-score. A Z-score expresses how many standard deviations an item deviates from the mean (Field 2017). To determine the strongest deviating traces, we assign this maximum deviation score as the final score for this pattern to each trace. The scoring of pattern 4 is similar, although it results in a value for each data field instead of each activity.

Patterns 6, 7, 10, and 12 result in a value for each trace. We calculate the mean of the values of all traces. We can then determine the Z-score for each trace, which is the score for this pattern.

Patterns 1 and 3 detect repeated events. Since more repeats of a single event or event sequence do not necessarily increase the chance of a workaround, we do not use this frequency for ranking traces. However, if there are few repeats in the dataset, we can evaluate all traces where at least one repeat occurs.

As mentioned before, we can only detect deviations in pattern 8 if we consider at which moments all events of a single activity occur. Since we work with timestamps for this, we cannot simply create a mean to compare to. Instead, we show the distribution of the timestamps as seen in Fig. 5.

We can create a ranking of the traces per pattern by sorting the related scores. A high-ranked trace may be more likely to contain a workaround, but since a workaround requires intent to reach a goal, which is rarely registered in an event log, it needs to be evaluated by an expert. Experts may not be able to provide the complete reasoning around the process; however, they are able to determine the intended end goal for a trace. The details of such an evaluation can be found in Sect. 5.

Table 5 Ranking of the traces using Z-scores. Sorted on pattern 2. Higher values for each trace indicate a stronger deviation from the mean

Trace	Pattern 2	Pattern 5a	Pattern 5b	...
T-023	8.186	0.847	1.821	...
T-104	8.186	1.332	1.523	...
T-243	8.186	0.847	0.846	...
T-247	8.186	0.847	0.226	...
T-281	8.186	0.847	0.505	...
T-219	7.446	2.070	1.442	...
...	...	...	...	...

4.5 Application

To summarize, we can apply the framework to an event log by following Fig. 7. We start by selecting the applicable patterns (Sect. 4.2), and then we apply the selected patterns (Sect. 4.3), which results in rankings of the traces for all patterns (Sect. 4.4). This ranking is then evaluated by a domain expert, which we describe next in Sect. 5.

5 Multiple-Case Study

5.1 Setting

The SWORD framework consists of both the patterns and the pattern selection process, in which the applicability of the patterns is directly dependent on the available data. To evaluate the framework, we performed a multiple-case study in two different departments of the same hospital: the emergency room (ER) and the cerebrovascular nursing ward. These departments handle very different processes. The ER mostly handles patients with strongly varying complaints for a short time and therefore needs to work with flexible processes, whereas patients on the ward get more specific care, often for a longer period. Because of these different process characteristics, we obtain a more complete view to evaluate the framework.

Both case studies were carried out at the University Medical Center Utrecht (UMCU), a Dutch academic hospital. The hospital takes care of more than 220,000 patients annually and has around 12,000 employees. Since this study makes use of patient data, it was evaluated and approved according to the Medical Research Ethics Committee (MREC) NedMec policy (research protocol number 22/1055).

In consultation with a data manager, we selected one week representative of typical hospital work. Thus we avoided the data being influenced by uncommon patient loads such as during holidays or centered around large-scale events like an outbreak of a specific disease.

5.2 Data Collection

The HIS stores all data in hundreds of separate tables. To identify the tables that contain the data that reflects the actions performed on patients in each department, we consulted a data manager of the UMCU with experience in extracting similar data from the HIS.

The ER uses a subsystem dedicated to track all its patients. This system assigns an identifier to each unique visit and keeps track of specific information, such as current patient location, the severity of the issue, and whether the patient was seen by a physician and a nurse. Especially the mutation log for this table provided a lot of events about the ER patients, as the table contained a timestamp for all activities and a resource for all except the 'seen by nurse/physician' activities. Since patients rarely stay longer than 24 h in the ER, we selected all traces that started within the selected week.

The ward does not use a ward-specific system, but patients can be tracked in a so-called admissions table. This logs to which ward patients are admitted, as well as a timestamp and executing resource for admittance, discharge, and transfers between wards. By filtering for the related ward in this table, we were able to extract which patients were at the ward during the specific week. Only selecting patients with a stay fully within the selected week would not have given a representative view, since many patients on this ward stay for multiple weeks. Therefore, we selected all hospitalizations at the ward during the selected week.

For both the ER and the ward, the HIS uses additional tables to keep track of what happens with patients. After consulting the data manager, we used the measurements table and the orders table. The measurements table logs the values of clinical measurements, such as heart rate and blood pressure, as well as pain scores, early warning scores, and other form-based measurements. For each measurement, the measurement time, the logging time, and the executing resource are registered. The orders table contains information about various orders. Among others, these can be orders for blood tests, radiology scans, or reminders for various other tasks. The logging time, planned time for the execution of the order, and resource are all registered. Note that to respect patient privacy, we do not use values or results from these tables.

We used Microsoft SQL to capture the event data from the HIS. Patients who object to their data being used for research purposes were tracked and removed from the data. One of the 32 patients in the ward and two of the 343 patients in the ER were filtered out for this reason. We pseudonymized the data as early as possible by assigning random unique values to all patient, resource, and other identifiers.

We combined the data into one event log for each department. The event log of the ER contained 7764 events

Fig. 7 The required steps to apply the SWORD framework

over 341 unique cases; the log of the cerebrovascular nursing ward had 2148 events over 31 cases.

5.3 SWORD Framework

To apply the SWORD framework, we first determined the patterns that apply to the data. By investigating the event logs created in the previous section, we found that we could use nine of the patterns, namely '*Activity frequency out of bounds*', '*Occurrence of a directly repeated activity*', '*Event executed by too multiple resources*', '*Event executed by a single resource*', '*Occurrence of activity outside of time period*', '*Delay between start of trace and event is out of bounds*', '*Time between event is out of bounds*', '*Duration of trace is out of bounds*', and '*Delay between event and logging is out of bounds*'. The remaining patterns require data that was not available, such as a process model or start- and end-times for each activity.

Applying these patterns as described in Sect. 3 results in a ranking of all traces for each pattern, with the higher-ranking ones deviating more strongly from the norm. The higher-ranked traces were therefore considered to be more likely to contain a workaround. This ranking was used to structure the evaluation of workarounds in these traces, which we explain further in the next section.

5.4 Evaluation

To evaluate the top-ranked traces, we conducted semi-structured interviews with one expert per department. These were recorded and later transcribed. The ER expert was a specialist in acute internal medicine. For the cerebrovascular nursing ward, we interviewed a neurologist.

For each of the patterns, we followed a similar approach during the interview. If a pattern provided a ranking of the traces, we selected the top traces to present to the expert. For most patterns, this was an overview of the top 30. Since the ward had 31 patients in total, patterns that result in a single measure per trace, such as 'Duration of trace out of bounds', only had 31 measures. Since presenting the top 30 traces would give an overview of almost all data, instead of the most deviating traces, we presented the top 6 (20%) instead. For these top-ranked traces, we presented simple, understandable measures for each pattern as described in Sect. 3, namely the mean and standard deviation of the top-ranked traces compared to those of all traces. If the patterns did not provide a ranking, we presented the corresponding figures instead. The expert was asked to indicate if the metrics pointed to a workaround or something else. If necessary, there was a short exchange between the expert and the researcher to determine what was happening in the trace.

In addition, we presented the highest-ranked trace for each pattern to the experts. We explained what happened during that trace and asked whether there was a workaround present in the trace. One trace was too long to present in this way (286 events). The trace concerned the delay between taking and logging measurements. Since they are often taken at the same time, we presented batches as a single event.

For example, Pattern 5 would be presented as follows: 'Seen by physician' usually occurs 36 min (SD = 101 min) after the patient arrives, but in the top-ranked traces, this happens after 71 h (SD = 5 h). The expert would then determine if the deviation was normal, a workaround, or something else.

5.5 Results

After ranking the traces using the patterns of the SWORD framework, we found at least one workaround for *each* pattern at the ER. With the nine patterns we were able to use, we found seven unique workaround types. In five of those workarounds, the overview of the top patterns was sufficient for the expert to recognize the behavior. They recognized the other two workarounds by inspecting the (pseudonymized) full, most-deviating trace. It should be noted that we detected three workaround types multiple times using different patterns. Table 6 shows an overview of the number of discovered workarounds per pattern.

For the ward, we found four unique workaround types. The expert recognized workaround behavior for three based on the overview of the top-ranked traces and one while investigating a most deviating trace.

The patterns in the framework are grouped into four main workaround dimensions (Mannhardt et al. 2015; Beerepoot et al. 2021): Control-flow, Data, Resource, and Time. To show that various workaround types can be discovered, we present an example for each dimension for both departments. Note that we could not use any of the Data-dimension patterns because of the requirements as described in Sect. 3.

5.5.1 Control-Flow Perspective

We detected three workaround types with our patterns from the control-flow perspective; two were found in the ER and one in the ward.

Example ER 'Activity frequency out of bounds' - The expert indicated that if the 'seen by nurse' activity did not occur in a trace, this pointed to a workaround. If a patient was not seen by a nurse, there are two options: (1) the patient left before this could happen, or (2) the patient was only administratively entered at the ER. Our expert indicated that the second option can happen when a patient is

referred to a ward for treatment by an outside party but does not have a patient number. New patients are difficult to register during weekends, except in the ER, since unexpected patients are handled by design. The new patients are then administratively admitted through the ER to get a patient number and get assigned to the ward immediately. Although it looks like the patients visited the ER, they never do: The functionality of the ER is merely used to work around the obstacle of registering patients during weekends.

Example Ward 'Directly Repeated Event' - According to the expert, the 'save treatment overview' activity should not reoccur directly in normative traces. When a nurse registers or updates the treatment overview of a patient, they should write this text themselves. Instead, they often open an old overview to copy (parts of) the text. Opening and closing the overviews would register in the system as saving the document multiple times, which shows this workaround.

5.5.2 Resource Perspective

We detected two workaround types with patterns from the resource perspective, both in the ER.

Example ER 'Activities executed by multiple resources' - After seeing the high number of resources in the most extreme trace, the expert explained that this should not occur normally. In some cases, there are more nurses involved than usual in moving patients between treatment rooms in the ER and taking measurements. These data should be immediately registered. However, especially when the ER is very busy, often one nurse checks the measured data and another nurse registers them in the HIS.

5.5.3 Time Perspective

We detected 13 workaround types with patterns from the time perspective; 10 were found in the ER and three in the ward.

Example ER 'Duration of Trace out of Bounds' (Note that this example was also detected with other patterns) - The expert indicated that an extremely long trace with a duration of more than three days would most likely contain a workaround. Patients always leave the ER within 24 h, usually even much faster. Before patients can be discharged in the system, there are checks that need to be performed, usually by a nurse. Some checks are not relevant for the illness, e.g., a pain score for a patient who has troubled breathing, but they are still enforced by the system before they can be officially discharged. If the ER is busy, nurses don't have time for these checks, but patients get sent home regardless. Since they cannot be removed from the system, staff register these patients in a field meant to keep track of patients that are sent somewhere else in the hospital with the label 'Administrative' until these checks can be logged at a later time. See Fig. 8 for a screenshot of the ER interface.

Example Ward'Time between Activities out of Bounds' - The expert explained that a long interval between a patient's 'arrival at the ward' and 'change room' pointed to a workaround. When treatment has stopped because there is no chance of recovery for the patient, the patient is not unnecessarily disturbed and moved to a private room. When there are many patients on the ward and there are few private rooms available, patients can get assigned to special rooms that are not used at the time, for example, a cleanroom that minimizes chances of infection. This is not an issue, but a workaround may occur when this happens and the special room needs to be used. Then, the patient may be moved to another room anyway. While that room is free and all care can technically continue, this does disturb the terminal patient.

5.6 Threats to Validity

To ensure construct validity, we need to establish the correct operational methods for the concepts under study (Yin 2009). The main threat to construct validity is

Table 6 Detected workarounds per pattern

Type	Pattern	ER	Ward
Control-flow	Activity frequency out of bounds	1	
	Occurrence of a directly repeated activity	1	1
Resource	Event executed by too many or too few resources	1	
	Event executed by too many or too few resources (accounted for trace length)	1	
Time	Occurrence of activity outside of time period	1	1
	Delay between start of trace and event is out of bounds	4	
	Time between events out of bounds	2	1
	Duration of trace out of bounds	2	
	Delay between event and logging is out of bounds	1	1

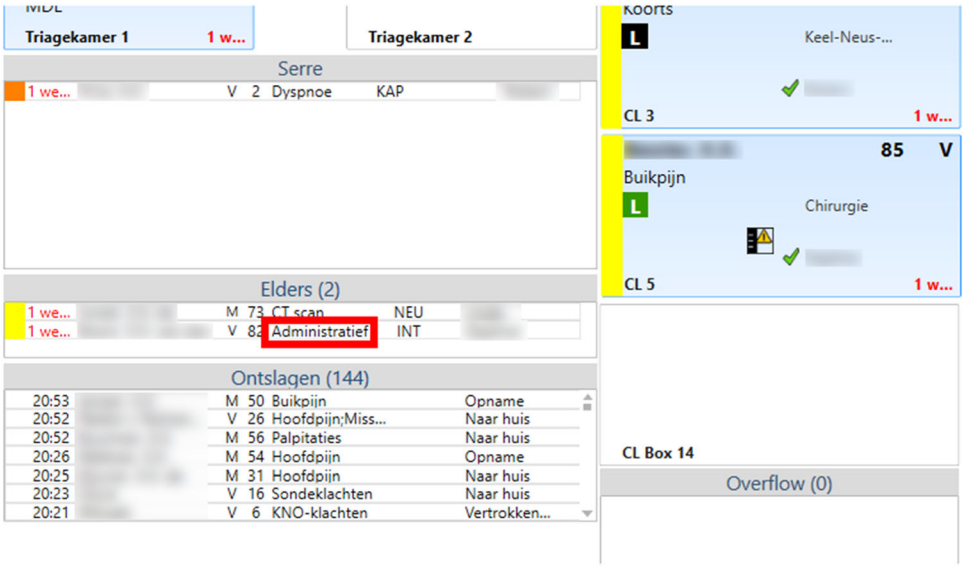

Fig. 8 Part of the ER interface that keeps track of patients, with all identifiable information blurred. The left side is used to track patients outside of specific ER rooms. From top to bottom: 'Serre' is a *waiting room*. The field 'Elders' (*Somewhere else*) is used to track patients that are somewhere else in the hospital (e.g., for a CT scan). 'Ontslagen' keeps track of the patients that were *discharged*. The right side shows patients in ER rooms. If patients are sent home, but not all required checks are done, they are temporarily put in the 'Elders' field and marked with 'Administratief' (*Administrative*)

related to the implementation of the patterns, namely that our code may not reflect the patterns sufficiently. Since the complexity of all implemented patterns is low, this threat is limited, but as a safety measure our code is publicly available.[2]

In this study, we try to generalize the use of the SWORD framework to general workaround discovery. A traditional threat to this type of generalization is that the results might only apply to the investigated organization (Yin 2009). Most of the patterns in the framework are based on existing process mining literature. Besides healthcare, these patterns originate from many different domains, such as education, finance, industry, insurance, logistics, or public administration (Di Francescomarino et al. 2018; Taymouri et al. 2021; Nguyen et al. 2016) and should therefore be applicable to multiple domains.

The newly defined patterns of the SWORD framework are based on 1) workarounds found in *healthcare* organizations other than our case organization and 2) a set of workarounds found in four *retail* organizations. Since the same workarounds tend to occur in multiple healthcare organizations (Beerepoot et al. 2021), we believe the newly defined patterns should be easily applicable in the wide field of healthcare. However, we see several similarities between the investigated domains that challenge further generalizability. First, it is worth mentioning that both fields contain strong hierarchical relationships between workers. Nurses and physicians in healthcare have a different standing, which is similar between management and salesperson roles in retail (van de Weerd et al. 2019). As the role of power has been shown to influence the emergence of workarounds (Beerepoot et al. 2019a), it may not be possible to apply our results to fields where there is little hierarchy present. For a more complete view, domains with less hierarchical forms of organizing should be investigated, such as startups as well as certain research and development firms. Second, all case studies have been conducted in the Netherlands. Other, non-Western, cultures may have different cultural dimensions, such as Power Distance, Uncertainty Avoidance, or Individualism (Hofstede 2011). Since these differences may affect workarounds, we cannot transfer outcomes to non-Western cultures at this point. Finally, we would like to note that none of the patterns in the time perspective were found in the workarounds from the retail domain. Although the framework in general would be applicable, it is possible that not all perspectives may be informative in every case.

There are also two differences between the domains that positively influence generalizability: 1) Retail work, specifically stocking and sales, tends to be relatively straightforward, while the processes in medical work may be highly complex (Safadi and Faraj 2010). 2) As opposed to retail, the healthcare domain is known to be knowledge-intensive, which influences processes in general (Ciccio

[2] https://git.science.uu.nl/w.g.waal/sword-framework

et al. 2012). Because of these differences, we find it likely that even though our case study is focused on healthcare, our results can be generalized to apply to multiple domains, within the constraints we mentioned.

To ensure that this study could be repeated, we saved all MS SQL queries that were used to extract the data from the database. With access to the database, these can be employed to extract the same datasets as used in this study, if the data has not been changed. As an extra safety measure, we also saved a frozen copy of the data. As mentioned before, the code to detect and rank the patterns in the datasets is publicly available. Finally, we recorded and transcribed both evaluation interviews.

5.7 Conclusion

In this multiple-case study, we applied nine patterns from the SWORD framework to data from the ER and a nursing ward of the UMCU. This resulted in a ranking of all traces for each pattern. We discussed the top-ranked traces with experts to evaluate the effectiveness of the framework. In total, we discovered 11 unique workaround types over the three investigated workaround perspectives.

6 Discussion

In this study, we described and evaluated the SWORD framework. We applied workaround detection patterns from the framework in a multiple-case study and found that many of the top-ranked traces indeed pointed to actual workarounds. In this section, we will discuss the scientific and practical implications of our work, as well as its limitations.

6.1 Scientific Implications

Compared to existing process mining approaches (Beerepoot et al. 2021; Outmazgin and Soffer 2014), we have discovered completely new workaround types, as opposed to detecting new instances of known types or discovering new ones using qualitative methods. In our quantitative approach, we start by investigating already existing data. This mitigates the chance of people exhibiting non-standard behavior, which can happen during interviews and observations (Holden 2001). Thus, our data-driven approach reflects the real process more objectively than the traditional methods. Compared to a quantitative neural network approach (Weinzierl et al. 2020), our patterns have simpler data requirements. This is underlined by the data from the investigated departments. The event log for the ER contained 341 cases, while the log for the nursing ward contained only 31. With this study, we show that we can indeed discover workarounds with limited data. The framework also does not require data to be labeled as a workaround or normative process beforehand, which is useful in scenarios where access to this data is limited.

Our scientific contributions can be extended from the workaround domain to the wider process mining domain. While the purpose of our framework is to discover workarounds, the patterns in the framework may also be applied to other fields, such as conformance checking (Dunzer et al. 2019), process variant analysis (Taymouri et al. 2021), predictive process monitoring (Di Francescomarino et al. 2018), and deviance mining (Nguyen et al. 2016). Patterns that originate from one field may be effective in the other fields. In addition, newly defined patterns may indicate new directions to conform to, deviate from, or monitor, depending on the field. In short, whenever one is interested in finding differences between processes, if an event log is available, the patterns may be applied to discover these differences.

In addition, we offer a framework that can be used to uncover and explain, at least partly, the dynamics of routines. Routine dynamics (cf. Feldman and Pentland (2022)) have a strong tradition in qualitative research, but with the advances in data science, it is possible to analyze event data to uncover them. Specifically, we are able to uncover their performative aspects, i.e., the specific actions people take, which might differ from the ostensive aspects of routines, i.e., what people understand the process is (Feldman and Pentland 2003). For example, workarounds can be used as an indication of resistance behavior (Wolf and Beverungen 2019). The ability to uncover workarounds with our framework opens up opportunities to study resistance and change.

Our work also furthers the conceptualization of workarounds. While Ejnefjäll and Ågerfalk (2019) show that multiple definitions of the term "workaround" are used in existing literature, we argue that the *intent to reach the goal* is essential to distinguish between workarounds and other behavior. For example, we present a nurse prescribing ad-hoc medication to a patient as a workaround, because they do not have time to wait for a physician. In this case, the *goal* is the same as waiting, i.e., prescribing medication before administering it. However, the *path* is different. If a nurse would forget the prescription and administer the medication immediately, the goal would accidentally be different; It would now only be administering medication. While this is a deviation from the designed process, this would be considered a mistake instead. Finally, a nurse could also intentionally change the goal, by aiming to use specific medication as much as possible for financial gain. This hypothetical scenario would clearly be considered fraud instead of a workaround. If the goal is the same, but only the path differs, this should

be considered a workaround, but once one attempts to reach a different goal, a different term should be used. Unintended deviations from the goal can be considered mistakes, but if the worker intentionally aims towards a different goal, this may be considered deception or even fraud.

6.2 Practical Implications

In addition to the scientific contributions, there are additional practical aspects of the framework: it is easy to apply and it requires limited effort from experts.

For process analysts, applying the SWORD framework is simple enough not to require domain knowledge until the evaluation step. Starting from an event log containing traces of only a single week, selecting and using the patterns is straightforward. Since most patterns result in a clear ranking of all traces, it is easy to select traces to evaluate with an expert. This shows that the framework may also be applied outside of research in a more practical scenario, even if there is little data available.

An important motivation for the quantitative approach of the SWORD framework is the limited time that domain experts need to spend on it compared to the traditional qualitative approaches. The bulk of the time for this study was spent on data since both extraction and preparation are difficult for hospital data. After implementing the patterns, using them to identify workarounds only takes a few moments. The domain experts are not required until after this step. This has vastly less of an impact on them compared to observations and interviews. Where previous studies making use of these qualitative measures required between 20 and 30 h of direct expert involvement for a single case study (Beerepoot and van de Weerd 2018; Beerepoot et al. 2019b), this study only required a single one-and-a-half-hour interview per expert for each case.

After this study, a process analyst from the UMCU expressed interest in investigating the implementation of the framework in their systems for more structural workaround detection. While an expression of interest is still far from actual application, it shows that this study, and workaround mining in general, is practically interesting to the healthcare sector. The newly discovered workarounds may be used to gain insights into processes and potentially improve them (Beerepoot and van de Weerd 2018).

6.3 Limitations

This study has the following limitations. First, the way we construct the ranking of the traces is somewhat limited. Not all traces that deviate from the norm necessarily point to workarounds. To the best of our knowledge, there is no previous work on this in workaround mining, so we opted for this simple ranking. Nevertheless, we described the procedure in detail in Sect. 4.4. In addition, we do not claim that the high-ranked traces indicate workarounds but instead use them as a starting point for the interviews.

Second, we have only interviewed a single expert per ward. While the physicians could easily judge traces involving the process they perform themselves, this was more difficult if the traces only concerned other resource types. Interviewing additional domain experts, such as nurses, would most likely result in more discovered workarounds. However, since we were more interested in investigating if we could discover workarounds and not so much in getting a complete view of their occurrence in a ward, the effect on this study is negligible.

Finally, it is important to note that this framework can only discover workarounds that are in the data. For example, from interviews, we know that nurses sometimes call the pharmacy to check if a prescription has arrived, instead of checking this in the system. Neither the call nor the check is registered in the data, so while interviews and observation can be used to detect these, there is no way to detect such workarounds using the framework or with any method that solely considers data from the system.

7 Conclusion

In this paper, we have described the SWORD framework and evaluated it. With a multiple-case study in a healthcare setting, we have discovered workarounds in two departments. We show that without much domain knowledge, we can use the framework to identify traces that point to workaround behavior. This requires limited effort from experts and can both identify known and discover unknown workaround types.

There are multiple research directions we would like to explore in further work. First, we make use of simple Z-scores for the current ranking of the patterns. However, we found that for most patterns only deviations higher than the mean were reflected in the top-ranked traces. While these deviations are interesting, we mostly found exceptionally high deviations and few low ones. The underlying data may be skewed because for most patterns there is a natural zero point, but no maximum value. For example, the duration of a trace can never be lower than zero, but a trace can always take longer. Using standard scores instead of Z-scores would already improve this, but more advanced statistics may be applicable too.

Second, in our study, we rank the likelihood of a workaround in the traces, but we only consider the top-ranked ones per pattern. We have seen that the patterns that lead to workarounds differ per ward, i.e., not all patterns

point to workarounds in all contexts. For future work, we would like to look further into which patterns are the most interesting depending on the context.

Third, while we have investigated the 13 patterns that may be used without domain knowledge, we were only able to apply nine. The data we used for this study was insufficient to test the remaining four patterns, but we would like to investigate these patterns in the future. This would require a dataset containing data lacking in this study. As noted earlier, there may be differences in which patterns are applicable in different cases. Future research could investigate this both within and between domains.

Fourth, the SWORD framework uses the patterns as singular options to detect differences between traces, but we have seen that some workarounds can be detected with multiple patterns. To improve detection, we could use machine learning methods, such as classification (Osuna et al. 1997) or clustering (Ester et al. 1996). These can combine detection patterns, allowing us to effectively consider processes from multiple angles at the same time.

Finally, the ability to discover more workarounds in a limited time can be leveraged to further research into workarounds. Since workarounds discovered using the SWORD framework can be found in data, they can be monitored over time to see how they originated, spread through an organization, or evolved.

Acknowledgements This publication is part of the WorkAround Mining (WAM!) project with project number 18490 which is partly financed by the Dutch Research Council (NWO). We would like to thank M.C.H. de Groot, PhD for his help during data extraction, as well as B. Vrijsen, MD and L.G. Exalto, MD, PhD for their time to evaluate our findings.

References

Alter S (2014) Theory of workarounds. CAIS 34:1041–1066

Azad B, King N (2008) Enacting computer workaround practices within a medication dispensing system. Eur J Inf Syst 17(3):264–278

Batista E, Solanas A (2018) Process mining in healthcare: a systematic review. In: 2018 9th international conference on information. intelligence, systems and applications (IISA), pp 1–6

Beerepoot I, Koorn J, van de Weerd I, van den Hooff B, Leopold H, Reijers HA (2019a) Working around health information systems: the role of power. In: Fortieth international conference on information systems (ICIS)

Beerepoot I, Lu X, van de Weerd I, Reijers HA (2021) Seeing the signs of workarounds: a mixed-methods approach to the detection of nurses' process deviations. In: Proceedings of the annual Hawaii international conference on system sciences (HICSS), pp 3763–3772

Beerepoot I, Ouali A, van de Weerd I, Reijers HA (2019b) Working around health information systems: to accept or not to accept? In: Twenty-seventh European conference on information systems

Beerepoot I, van de Weerd I (2018) Prevent, redesign, adopt or ignore: improving healthcare using knowledge of workarounds. In: Proceedings of the 26th European conference on information systems (ECIS), pp 1–15

Bolt A, de Leoni M, van der Aalst WMP (2018) Process variant comparison: using event logs to detect differences in behavior and business rules. Inf Syst 74:53–66

Borrego D, Barba I (2014) Conformance checking and diagnosis for declarative business process models in data-aware scenarios. Expert Syst Appl 41(11):5340–5352

Bose RPJC, van der Aalst WMP (2013) Discovering signature patterns from event logs. In: IEEE symposium on computational intelligence and data mining (CIDM), pp 111–118

Burattin A, Maggi FM, Sperduti A (2016) Conformance checking based on multi-perspective declarative process models. Expert Syst Appl 65:194–211

Ciccio CD, Marrella A, Russo A (2012) Knowledge-intensive processes: an overview of contemporary approaches. In: 1st international workshop on knowledge-intensive business processes (KiBP 2012) pp 33–47

de Leoni M, van der Aalst WMP (2013) Aligning event logs and process models for multi-perspective conformance checking: an approach based on integer linear programming. Lect Notes Comput Sci 8094:113–129

Di Francescomarino C, Ghidini C, Maggi FM, Milani F (2018) Predictive process monitoring methods: which one suits me best? In: Business process management (BPM), pp 462–479

Dunzer S, Stierle M, Matzner M, Baier S (2019) Conformance checking: a state-of-the-art literature review. In: Proceedings of the 11th international conference on subject-oriented business process management (S-BPM), pp 1–10

Ejnefjäll T, Ågerfalk PJ (2019) Conceptualizing workarounds: meanings and manifestations in information systems research. CAIS 340–363

Ester M, Kriegel HP, Sander J, Xu X (1996) A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the 2nd international conference on knowledge discovery and data mining (KDD), pp 226–231

Feldman MS, Pentland BT (2003) Reconceptualizing organizational routines as a source of flexibility and change. Admin Sci Q 48(1):94–118

Feldman MS, Pentland BT (2022) Routine dynamics: toward a critical conversation. Strat Organ 20(4):846–859

Field A (2017) Discovering statistics using IBM SPSS. Sage, Thousand Oaks

Hofstede G (2011) Dimensionalizing cultures: the Hofstede model in context. Online Read Psychol Cult 2(1):8

Holden JD (2001) Hawthorne effects and research into professional practice. J Eval Clin Pract 7(1):65–70

Ignatiadis I, Nandhakumar J (2009) The effect of ERP system workarounds on organizational control: an interpretivist case study. Scand J Inf Syst 21(2):3

Ilie V (2013) Psychological reactance and user workarounds. a study in the context of electronic medical records implementations. In: Proceedings of the 21st European conference on information systems (ECIS), p 24

Lazovik A, Aiello M, Papazoglou M (2004) Associating assertions with business processes and monitoring their execution. In: Proceedings of the 2nd international conference on service oriented computing (ICSOC), pp 94–104

Lo D, Cheng H, Han J, Khoo SC, Sun C (2009) Classification of software behaviors for failure detection. In: Proceedings of the 15th international conference on knowledge discovery and data mining (KDD), pp 557–566

Lu X, Fahland D, van den Biggelaar FJHM, van der Aalst WMP (2016) Detecting deviating behaviors without models. In: Business process management workshops, pp 126–139

Mahbub K, Spanoudakis G (2004) A framework for requirents monitoring of service based systems. In: Proceedings of the 2nd international conference on service oriented computing (ICSOC), pp 84–93

Mannhardt F, de Leoni M, Reijers HA, van der Aalst WMP (2015) Balanced multi-perspective checking of process conformance. Computing 98(4):407–437

Nguyen H, Dumas M, La Rosa M, Maggi FM, Suriadi S (2016) Business process deviance mining: review and evaluation. ArXiv e-prints arXiv:https://arxiv.org/pdf/1608.08252pdf, accessed 20 Nov 2023

Osuna E, Freund R, Girosi F (1997) An improved training algorithm for support vector machines. In: Neural networks for signal processing VII. proceedings of the 1997 IEEE signal processing society workshop, pp 276–285

Outmazgin N, Soffer P (2014) A process mining-based analysis of business process work-arounds. Softw Syst Model 15(2):309–323

Ras G, van Gerven M, Haselager P (2018) Explanation methods in deep learning: users, values, concerns and challenges. In: Explainable and interpretable models in computer vision and machine learning, pp 19–36

Rebuge Á, Ferreira DR (2012) Business process analysis in healthcare environments: a methodology based on process mining. Inf Syst 37(2):99–116

Röder N, Wiesche M, Schermann M (2014) A situational perspective on workarounds in IT-enabled business processes: a multiple case study. In: Proceedings of the 22nd European conference on information systems (ECIS), pp 1–15

Rozinat A, van der Aalst WMP (2008) Conformance checking of processes based on monitoring real behavior. Inf Syst 33(1):64–95

Safadi H, Faraj S (2010) The role of workarounds during an opensource electronic medical record system implementation. In: Thirty first international conference on information systems ICIS

Swennen M, Martin N, Janssenswillen G, Jans M, Depaire B, Caris A, Vanhoof K (2016) Capturing resource behaviour from event logs. In: Proceedings of the 6th international symposium on data-driven process discovery and analysis (SIMPDA), pp 130–134

Taymouri F, La Rosa M, Dumas M, Maggi FM (2021) Business process variant analysis: survey and classification. Knowl Based Syst 211(106):557

Tucker AL (2016) The impact of workaround difficulty on frontline employees' response to operational failures: a laboratory experiment on medication administration. Manag Sci 62(4):1124–1144

van der Aalst WMP, van Dongen BF, Herbst J, Maruster L, Schimm G, Weijters AJMM (2003) Workflow mining: a survey of issues and approaches. Data Knowl Eng 47(2):237–267

van de Weerd I, Vollers P, Beerepoot I, Fantinato M (2019) Workarounds in retail work systems: prevent, redesign, adopt or ignore? In: Proceedings of the 27th European conference on information systems (ECIS), pp 1–15

van der Waal W, Beerepoot I, van de Weerd I, Reijers HA (2022) The SWORD is mightier than the interview: a framework for semi-automatic workaround detection. In: Business process management (BPM), pp 91–106

Vogelsmeier AA, Halbesleben JRB, Scott-Cawiezell JR (2008) Technology implementation and workarounds in the nursing home. J Am Med Inform Assoc 15(1):114–119

Weinzierl S, Wolf V, Pauli T, Beverungen D, Matzner M (2022) Detecting temporal workarounds in business processes–a deep-learning-based method for analysing event log data. J Bus Anal 5(1):76–100

Weinzierl S, Wolf V, Pauli T, Beverungen D, Matzner M (2020) Detecting workarounds in business processes-a deep learning method for analyzing event logs. In: Proceedings of the 28th European conference on information systems (ECIS), pp 1–16

Wolf V, Beverungen D (2019) Conceptualizing the impact of workarounds: an organizational routines perspective. In: Proceedings of the 27th European conference on information systems (ECIS), pp 1–11

Wynn MT, Poppe E, Xu J, ter Hofstede AHM, Brown R, Pini A, van der Aalst WMP (2017) ProcessProfiler3D: a visualisation framework for log-based process performance comparison. Decis Support Syst 100:93–108

Yin RK (2009) Case study research: design and methods, vol 5, 4th edn. Sage, Thousand Oaks